
MAINCRAFTS TECHNOLOGY

IoT-Enabled Smart Lighting System with Blynk Cloud

Embedded Systems – Task 3

Project

**Automatic Light Detection System with Blynk IoT Integration
using ESP32**

Submitted by

Shree Harshan K

Date: 18-07-2026

Table of Contents

Table of Contents.....	2
1. Introduction	3
1.1 What is IoT?	3
1.2 What is Blynk?.....	3
2. Component Study	4
2.1 ESP32 DevKit V1	4
2.2 LDR Sensor Module	4
2.3 LED and 220Ω Resistor.....	4
2.4 Blynk IoT Platform	4
2.5 Wokwi ESP32 Simulator.....	4
3. Project Details.....	5
3.1 Project Title	5
3.2 Objective	5
3.3 Components Used.....	5
3.4 Circuit Diagram.....	5
3.5 Pin Connections	6
3.6 Blynk Virtual Pin Mapping.....	6
4. Working Principle	7
4.1 Automatic Mode.....	7
4.2 Manual Override Mode	7
4.3 Blynk Dashboard Updates	7
4.4 Serial Monitor Output.....	7
5. Blynk Dashboard Design.....	8
5.1 Dashboard Screenshot.....	8
6. Source Code with Explanation	9
6.1 Code Walkthrough.....	10
7. Results.....	11
8. Key Features.....	11
9. Real-World Applications	12
Smart Street Lighting with Remote Control.....	12
Industrial Facility Lighting	12
Smart Home Automation	12
Greenhouse and Agricultural Lighting.....	12
Security and Perimeter Lighting.....	12
10. Comparison: Task 2 vs Task 3.....	12
11. Conclusion	13

1. Introduction

Task 3 builds directly on the Automatic Light Detection System developed in Task 2. While Task 2 demonstrated standalone embedded automation — reading an LDR sensor and controlling an LED based on a threshold — it had no way of communicating its status to the outside world. The upgrade in Task 3 addresses this by integrating the ESP32 with the Blynk IoT cloud platform, transforming a closed embedded system into a connected smart device.

This is one of the most important shifts in modern embedded engineering: the move from simple embedded automation to IoT-enabled smart systems. Once a device can communicate over the internet, it becomes possible to monitor it remotely, receive live data on a dashboard, and even send commands back to the device from anywhere in the world.

1.1 What is IoT?

The Internet of Things (IoT) refers to a network of physical devices that are embedded with sensors, software, and communication hardware, allowing them to collect and exchange data over the internet. IoT devices range from simple smart plugs to complex industrial machinery. The defining characteristic is that these devices can be monitored and controlled remotely without physical access.

1.2 What is Blynk?

Blynk is an IoT platform designed to connect microcontrollers like Arduino and ESP32 to the cloud. It provides a mobile and web dashboard that allows users to visualise live sensor data, control output devices, and build interactive interfaces without needing to build a backend from scratch. Blynk handles the cloud infrastructure, authentication, and data routing, so the developer only needs to focus on what to display and what to control.

Blynk communicates with the ESP32 using a unique Template ID and Authentication Token, which are generated when a new device is registered on the Blynk platform.

2. Component Study

2.1 ESP32 DevKit V1

The ESP32's built-in Wi-Fi module is the core enabler of this task. Unlike the Arduino UNO, which requires an external shield to connect to the internet, the ESP32 can connect to a Wi-Fi network natively. It uses this connection to reach the Blynk cloud server and exchange data in real time. The dual-core architecture also allows the ESP32 to handle both sensor reading and cloud communication simultaneously without delays.

2.2 LDR Sensor Module

The same LDR sensor module used in Task 2 is retained here. It produces an analog output on its AO pin proportional to the ambient light intensity. The ESP32 reads this value on GPIO34 using `analogRead()` and uses it for both local automation decisions and cloud data transmission.

2.3 LED and 220Ω Resistor

The LED continues to serve as the output indicator, now controlled by both the automatic LDR-based logic and the manual override switch on the Blynk dashboard. The 220Ω resistor remains in series to protect the LED and the GPIO pin.

2.4 Blynk IoT Platform

Blynk acts as the communication bridge between the ESP32 and the user's dashboard. Data is sent to the cloud using virtual pins (V0, V1, V2, V3), which are software-defined channels that map to dashboard widgets. The Blynk library running on the ESP32 handles all the networking and protocol communication internally.

2.5 Wokwi ESP32 Simulator

Wokwi was used to design and test the circuit. It supports ESP32 simulation including Wi-Fi functionality, making it possible to test the Blynk cloud connection without physical hardware.

3. Project Details

3.1 Project Title

Automatic Light Detection System with Blynk IoT Integration using ESP32

3.2 Objective

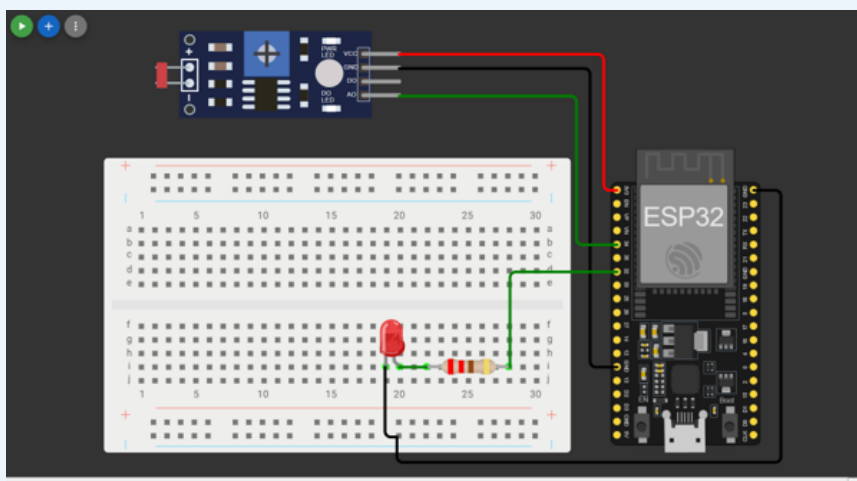
To upgrade the Task 2 Automatic Light Detection System by integrating ESP32 Wi-Fi connectivity and Blynk IoT cloud support, enabling real-time remote monitoring of sensor data and system status, and adding a manual override feature accessible from the Blynk web and mobile dashboards.

3.3 Components Used

- ESP32 DevKit V1 – microcontroller with built-in Wi-Fi
- LDR Sensor Module (Analog Output) – ambient light detection
- LED – output indicator
- 220Ω Resistor – current limiting
- Breadboard and Jumper Wires
- Blynk IoT Platform – cloud monitoring and control
- Wokwi ESP32 Simulator – simulation and testing
- Arduino IDE (Arduino Framework) – code development
- GitHub – version control and documentation

3.4 Circuit Diagram

The hardware circuit is identical to Task 2 since the IoT upgrade is achieved entirely through software. The Blynk cloud connection is established over the ESP32's built-in Wi-Fi and does not require any additional wiring.



3.5 Pin Connections

Component	ESP32 Pin
LDR Module – VCC	3.3V
LDR Module – GND	GND
LDR Module – AO	GPIO 34
LDR Module – DO	Not connected
LED – Anode (+)	GPIO 32 (via 220Ω resistor)
LED – Cathode (–)	GND

3.6 Blynk Virtual Pin Mapping

Virtual Pin	Purpose / Widget
V0	LDR Value – Gauge widget (displays raw ADC reading)
V1	Environment Status – Label widget (BRIGHT / DARK)
V2	LED Status – LED Indicator widget (ON / OFF)
V3	Manual Override – Switch widget (user-controlled)

4. Working Principle

On startup, the ESP32 initialises the serial monitor, configures the LED pin as an output, and then attempts to connect to the Wokwi Wi-Fi network. Once Wi-Fi is established, it connects to the Blynk cloud using the registered Template ID and Authentication Token.

Once connected, the system enters its main loop, which runs every second and performs three key jobs: reading the LDR sensor, making the LED control decision, and updating the Blynk dashboard.

4.1 Automatic Mode

In automatic mode, the ESP32 reads the LDR analog value from GPIO34 and compares it against the threshold of 2000. If the reading is below 2000, the environment is classified as DARK and the LED is switched on. If the reading is 2000 or above, the environment is BRIGHT and the LED is switched off. This is the same logic that was implemented in Task 2.

4.2 Manual Override Mode

The Blynk dashboard includes a Switch widget mapped to virtual pin V3. When a user enables this switch from the web or mobile dashboard, the `overrideState` flag in the code is set to `true`. In this state, the LED is forced ON regardless of what the LDR sensor reads, and the Serial Monitor reports 'Manual Override: ON'. Disabling the switch returns the system to automatic mode. This feature demonstrates bidirectional IoT communication: not just sending data to the cloud, but also receiving commands from it.

4.3 Blynk Dashboard Updates

After every LED control decision, the ESP32 pushes four data points to the Blynk cloud using `Blynk.virtualWrite()`:

- V0 – the raw LDR sensor value (displayed as a gauge)
- V1 – the environment status string (BRIGHT or DARK)
- V2 – the LED state (1 for ON, 0 for OFF)
- V3 – reflects the current manual override state on the dashboard

The `Blynk.run()` call at the end of each loop iteration keeps the cloud connection alive and processes any incoming commands from the dashboard, including manual override toggle events.

4.4 Serial Monitor Output

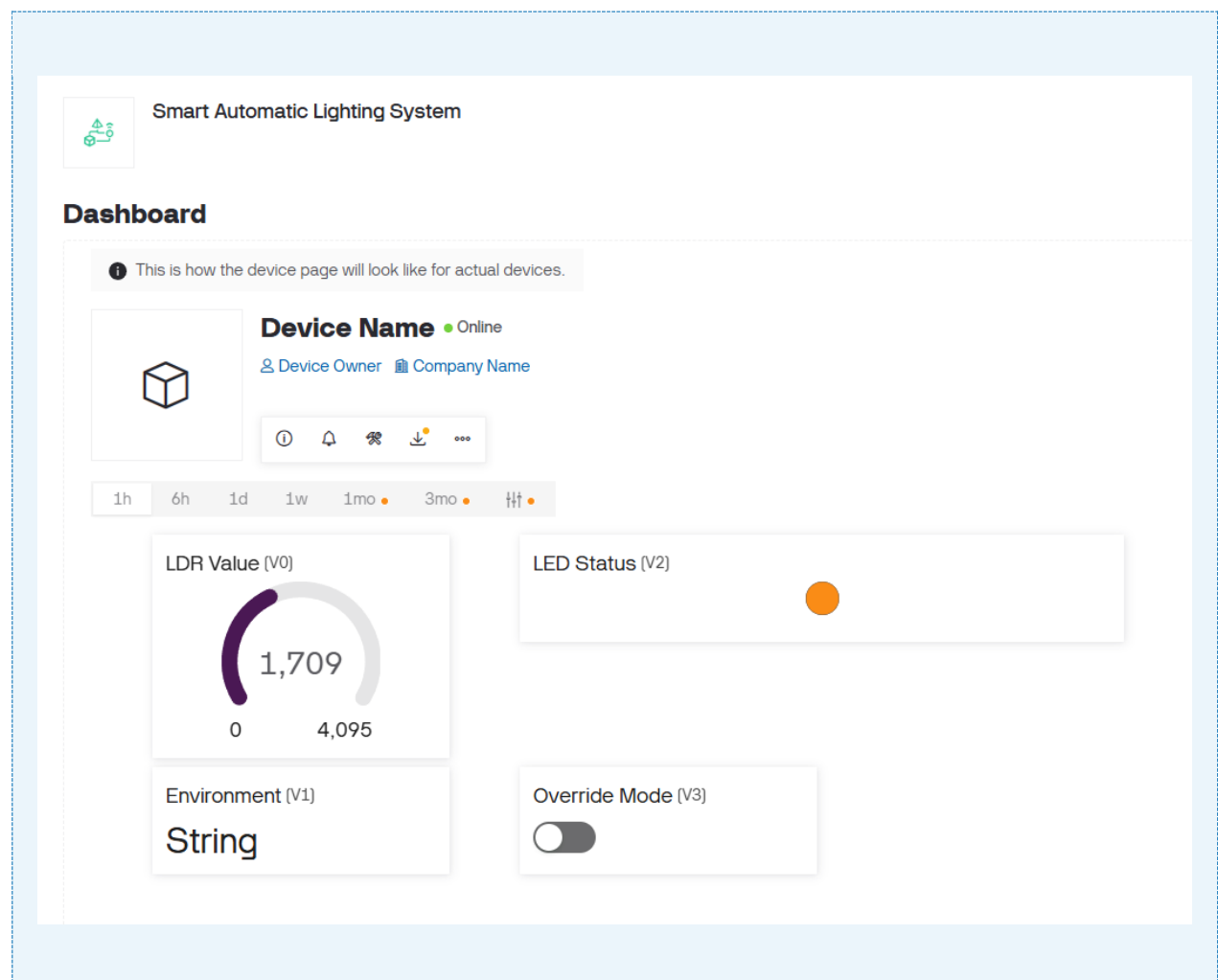
The Serial Monitor displays the following information every second: Wi-Fi and Blynk connection status, the LDR sensor value, environment status (BRIGHT/DARK), LED state, and manual override status. This provides a complete debugging view of the system during testing.

5. Blynk Dashboard Design

The Blynk dashboard was set up at blynk.cloud and configured with four widgets to visualise and interact with the system. Both a web dashboard and a mobile dashboard were created for this project.

Widget	Virtual Pin Purpose
Gauge	V0 Displays the live LDR sensor reading (0–4095)
Label	V1 Shows the environment status as text (BRIGHT / DARK)
LED Indicator	V2 Lights up on the dashboard when the physical LED is ON
Switch	V3 Manual override toggle – forces LED ON when enabled

5.1 Dashboard Screenshots



6. Source Code with Explanation

The following Arduino/C++ code adds Blynk cloud integration and manual override functionality on top of the Task 2 base logic. Each section is commented for clarity.

```
// — Library Includes —
// Define Blynk credentials BEFORE including BlynkSimpleEsp32.h
#define BLYNK_TEMPLATE_ID   "YOUR_TEMPLATE_ID"
#define BLYNK_TEMPLATE_NAME "Smart Light Monitor"
#define BLYNK_AUTH_TOKEN    "YOUR_AUTH_TOKEN"

#include <WiFi.h>           // ESP32 Wi-Fi library
#include <BlynkSimpleEsp32.h> // Blynk library for ESP32

// — Wi-Fi Credentials —
char ssid[] = "Wokwi-GUEST"; // Wokwi simulator network
char pass[] = "";            // No password needed on Wokwi

// — Pin and Threshold Definitions —
#define LDR_PIN      34    // LDR analog output to GPIO34
#define LED_PIN      32    // LED connected to GPIO32
#define LIGHT_THRESHOLD 2000 // Below = DARK, Above = BRIGHT

// — Global State Variable —
bool overrideState = false; // Tracks Manual Override status from Blynk

// — Blynk Callback: Manual Override Switch (V3) —
// This function is called automatically whenever the V3 switch is toggled
BLYNK_WRITE(V3) {
  overrideState = param.asInt(); // 1 = Override ON, 0 = Override OFF
}

// — setup() - runs once on power-up —
void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  // Connect to Wi-Fi and Blynk cloud
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
  Serial.println("Connected to Wi-Fi and Blynk!");
}

// — loop() - runs repeatedly —
void loop() {
  Blynk.run(); // Maintain Blynk cloud connection & handle incoming events

  int ldrValue = analogRead(LDR_PIN); // Read LDR sensor (0-4095)

  String envStatus;
  bool ledState;

  if (overrideState) {
    // — MANUAL OVERRIDE MODE —
    digitalWrite(LED_PIN, HIGH); // Force LED ON
    ledState = true;
    envStatus = (ldrValue < LIGHT_THRESHOLD) ? "DARK" : "BRIGHT";
    Serial.println("Manual Override: ON | LED forced ON");
  } else {
    // — AUTOMATIC MODE —
    if (ldrValue < LIGHT_THRESHOLD) {
```

```
    digitalWrite(LED_PIN, HIGH); // Dark - turn LED ON
    envStatus = "DARK";
    ledState = true;
  } else {
    digitalWrite(LED_PIN, LOW); // Bright - turn LED OFF
    envStatus = "BRIGHT";
    ledState = false;
  }
}

// — Push data to Blynk Dashboard —————
Blynk.virtualWrite(V0, ldrValue); // Gauge: raw LDR value
Blynk.virtualWrite(V1, envStatus); // Label: BRIGHT or DARK
Blynk.virtualWrite(V2, ledState ? 1 : 0); // LED widget: ON / OFF
Blynk.virtualWrite(V3, overrideState ? 1 : 0); // Switch: current state

// — Serial Monitor Output —————
Serial.print("LDR: "); Serial.print(ldrValue);
Serial.print(" | Environment: "); Serial.print(envStatus);
Serial.print(" | LED: "); Serial.print(ledState ? "ON" : "OFF");
Serial.print(" | Override: "); Serial.println(overrideState ? "ON" : "OFF");

delay(1000); // Update every 1 second
}
```

6.1 Code Walkthrough

The code begins by defining the three Blynk credentials — Template ID, Template Name, and Auth Token — before including the Blynk library. This ordering is required by the library. The WiFi.h library manages the physical connection, while BlynkSimpleEsp32.h handles all cloud protocol communication.

The BLYNK_WRITE(V3) callback is a special function that the Blynk library calls automatically whenever the V3 switch on the dashboard is toggled. This is how the ESP32 receives commands from the cloud without needing to constantly poll for them. The value is stored in the overrideState boolean for use in the main loop.

In setup(), Blynk.begin() handles both Wi-Fi connection and Blynk cloud authentication in a single call. The main loop starts with Blynk.run(), which must be called regularly to keep the cloud connection alive and to process any pending incoming events such as override toggle commands.

The rest of the loop reads the LDR, applies either override or automatic logic to decide the LED state, then pushes all four data points to the dashboard using Blynk.virtualWrite(). The Serial Monitor output mirrors the dashboard, making it easy to verify the system's behaviour during testing.

7. Results

The system was tested in Wokwi with the Blynk cloud connected. The LDR sensor slider was adjusted manually to simulate bright and dark conditions, and the override switch was toggled from the Blynk dashboard to verify bidirectional communication.

Test Condition	Result
LDR $\approx$ 3017 (Bright), Override OFF	LED OFF Dashboard: BRIGHT, Gauge = 3017, LED widget OFF
LDR $\approx$ 32 (Dark), Override OFF	LED ON Dashboard: DARK, Gauge = 32, LED widget ON
LDR $\approx$ 3017 (Bright), Override ON	LED ON Dashboard: BRIGHT, Gauge = 3017, LED widget ON, Switch ON
LDR $\approx$ 32 (Dark), Override ON	LED ON Dashboard: DARK, Gauge = 32, LED widget ON, Switch ON
Override disabled after being ON	System returned to automatic mode immediately

All five test conditions produced the expected behaviour. The Blynk dashboard updated in real time without noticeable delay, and the override feature correctly overrode the automatic logic without interfering with sensor monitoring.

8. Key Features

- Automatic lighting control using an LDR sensor and IF-ELSE logic
- ESP32 Wi-Fi connectivity for cloud communication
- Real-time cloud monitoring via the Blynk IoT platform
- Web dashboard and mobile dashboard support
- Live LDR value display using a Gauge widget
- Environment status (BRIGHT/DARK) via a Label widget
- LED status indicator mirrored on the Blynk dashboard
- Manual Override feature enabling user-controlled LED activation from anywhere
- Continuous Serial Monitor output for local debugging and monitoring
- Fully simulated and verified using the Wokwi ESP32 platform

9. Real-World Applications

The combination of local automation and remote monitoring in this project makes it relevant to a wide range of real-world applications:

Smart Street Lighting with Remote Control

Municipal authorities can monitor the status of street lights remotely through a dashboard. If a light fails to respond correctly to ambient conditions, the override feature can be used to manually control it until a physical inspection takes place.

Industrial Facility Lighting

Large factories and warehouses can automate lighting across multiple zones while allowing facility managers to monitor energy usage and override specific zones from a central dashboard.

Smart Home Automation

Homeowners can monitor indoor and outdoor lighting conditions from their smartphone and override automatic control when needed, for example when hosting an event and requiring lights to stay on regardless of ambient brightness.

Greenhouse and Agricultural Lighting

Supplemental grow lights can be automatically activated in low-light conditions, while remote monitoring allows farmers to check and adjust lighting schedules from anywhere.

Security and Perimeter Lighting

Security personnel can remotely switch perimeter lights on or off through the Blynk dashboard, providing flexible control over illuminated zones without needing to be on-site.

10. Comparison: Task 2 vs Task 3

The table below summarises what was added in Task 3 compared to the Task 2 baseline:

Feature	Task 2 → Task 3
Sensor & LED control	Present in both tasks (unchanged)
Serial Monitor output	Present in both tasks (extended in Task 3)
Wi-Fi connectivity	Not present → Added in Task 3
Cloud monitoring dashboard	Not present → Added via Blynk
Real-time remote data display	Not present → Gauge, Label, LED widget
Manual override from cloud	Not present → Switch widget on Blynk
Bidirectional communication	Not present → BLYNK_WRITE callback

11. Conclusion

Task 3 represents a meaningful step up from the previous tasks. By adding Wi-Fi connectivity and Blynk cloud integration to the existing LDR-based light detection system, a basic embedded automation project was transformed into a full IoT-enabled smart device with remote monitoring and control capabilities.

The most valuable learning from this task was understanding how bidirectional communication works in IoT systems. Sending data to the cloud was straightforward using `Blynk.virtualWrite()`. Receiving commands back — through the `BLYNK_WRITE(V3)` callback — showed how an event-driven model works in embedded programming, where functions are triggered automatically by external events rather than being called from a fixed sequence in the main loop.

Designing the Blynk dashboard also reinforced the importance of the user interface in IoT projects. A system that collects and acts on sensor data is useful; one that presents that data clearly and allows human intervention from anywhere is genuinely practical. This project combines embedded programming, sensor interfacing, Wi-Fi communication, cloud integration, and UI design into a single working system, which is exactly what modern IoT engineering looks like.

Submitted by: Shree Harshan K

Task: Embedded Systems Task 3 – Maincrafts Technology

Date: 18-07-2026